

My Project

Generated by Doxygen 1.13.2

1 README	1
2 Hierarchical Index	5
2.1 Class Hierarchy	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Class Documentation	11
5.1 Student Class Reference	11
5.1.1 Member Function Documentation	12
5.1.1.1 test()	12
5.2 Vector< T > Class Template Reference	12
5.3 Zmogus Class Reference	13
6 File Documentation	15
6.1 lib.h	15
6.2 V1.5vect.h	15
Index	21

Chapter 1

README

Naudojimosi instrukcijos: Norint paleisti programą, į terminalą įvesti "make run_test" Norint ištrinti sukompiliuotus failus, į terminalą įvesti "make clean"

Ši programa leidžia:

Įvesti studentų duomenis rankiniu būdu, automatiškai arba iš failo Apskaičiuoti galutinį pažymių rezultatą pagal vidurkį arba medianą Skirstyti studentus į dvi grupes: pažangius ir nepažangius pagal dvi strategijas Rezultatus išvesti į ekraną arba į failus alfos.txt ir susmukeliai.txt Rūšiuoti studentus pagal vardą, pavardę arba galutinį rezultatą

Programa naudoja dvi klases. Bazinę Žmogaus klasę, kuri aprašo bendrus žmogaus atributus (vardas, pavardė). Turi gryną virtualią funkciją test(), todėl iš jos negalima kurti objektų. Ir klasę Studentas, kuri yra paveldima iš Žmogaus klasės. Joje saugomi Studento pažymiai ir galutinis rezultatas. Implementuoja test() metodą, galima kurti Studento tipo objektus.

Perdengtų metodų paaiškinimas

----- Įvesties būdai -----

Tipas	Realizacija	Aprašymas
Rankinis	pasirinkimas == 1	Vartotojas suveda vardą, pavardę, ND ir egzaminą
Pusiau auto	pasirinkimas == 2	Automatiškai generuojami tik pažymiai
Automatinis	pasirinkimas == 3	Visi duomenys sugeneruojami automatiškai
Iš failo	failo pasirinkimas == 1,2,3,4,5	Visi duomenys sugeneruojami kursiškai.txt faile

----- Išvesties būdai -----

Tipas	Realizacija	Failai
Ekranas	pasirinkimas == 1, spausdinam()	Duomenys išvedami į ekraną pagal rikiavimą
Į failą	pasirinkimas == 2, spausdinam()	Sukuriami alfos.txt ir susmukeliai.txt

Vektoriaus funkcijų paaiškinimas:

1. `push_back` funkcija Paskirtis: Prideda naują elementą į vektoriaus pabaigą.

Veikimas: Jei nėra pakankamai vietos (t. y. talpa lygi dydžiui), talpa padvigubinama. Sukuriamas naujas masyvas su didesne talpa, į jį perkopijuojami esami elementai. Galiausiai pridedamas naujas elementas į galą.

1. `pop_back` funkcija Paskirtis: Pašalina paskutinį vektoriaus elementą.

Veikimas: Patikrina, ar vektorius nėra tuščias. Sumažina dydį vienetu.

1. `getSize` funkcija Paskirtis: Grąžina šiuo metu esančių elementų skaičių vektoriuje. Veikimas: Tiesiogiai grąžina `size` reikšmę.
2. `resize` funkcija Paskirtis: Keičia vektoriaus dydį į nurodytą.

Veikimas: Jei naujas dydis didesnis nei esamas, pridedami numatytieji elementai. Jei mažesnis – vektoriaus dydis sumažinamas. Jei reikia – padidinama talpa.

1. Konstruktorius ir destruktorius Paskirtis: Konstruktorius inicializuoja tuščią vektorių. Destruktorius atlaisvina užimtą atmintį.

Veikimas: Konstruktorius nustato `data = nullptr`, `size = 0`, `capacity = 0`. Destruktorius naudoja `delete[]` atlaisvinti dinamiškai priskirtą atmintį.

==== REZULTATŲ LENTELĖ ====

Veiksmas	V1.0 (1M)	V1.1 (1M)	V1.0 (10M)	V1.1 (10M)
Failo kūrimo laikas	3.15s	3.02s	29.61s	29.57s
Duomenų nuskaitymo laikas	7.54s	8.08s	76.73s	82.71s
Skirstymo laikas	0.84s	1.14s	9.70s	12.18s
Rezultatų išvedimo laikas	0.40s	0.39s	3.96s	4.25s
Rūšiavimo tvarka laikas	0.04s	0.18s	0.45s	1.69s
-----	-----	-----	-----	-----
Visos programos laikas	18.68s	21.86s	136.59s	146.66s

==== FLAGŲ PALYGINIMAS ====

Veiksmas	V1 -O1	V1 -O2	V1 -O3	V1.1 -O1	V1.1 -O2	V1.1 -O3
Failo kūrimo laikas	2.96s	2.99s	3.00s	2.97s	3.01s	2.97s
Duomenų nuskaitymo laikas	7.48s	7.51s	7.53s	8.09s	8.09s	8.09s
Skirstymo į blogus/gerus laikas	0.84s	0.83s	0.83s	1.14s	1.15s	1.15s
Rezultatų išvedimo laikas	0.40s	0.40s	0.40s	0.42s	0.40s	0.45s
Rūšiavimo tvarka laikas	0.03s	0.03s	0.04s	0.18s	0.18s	0.18s
-----	-----	-----	-----	-----	-----	-----
Visos programos laikas	19.06s	19.32s	18.03s	18.01s	17.16s	17.05s

==== Vektorių `push_back` palyginimas ====

File	Custom Vector (laikas)	STL Vector (laikas)
1000.txt	0.01s	0.01s
10000.txt	0.03s	0.04s
100000.txt	0.13s	0.18s
1000000.txt	1.06s	1.54s
10000000.txt	11.74s	16.08s
-----	-----	-----

==== Vektorių veikimo laiko palyginimas ==== Custom vektorius

Failas	Kūrimo	Nuskaitymas	Skirstymas	Išvedimas	Rūšiavimas	Viso laikas
kursiokai100k	0.49s	0.95s	0.01s	0.06s	0.03s	13.83s
kursiokai1M	4.93s	9.65s	0.13s	0.57s	0.14s	21.76s
kursiokai10M	49.24s	98.74s	1.45s	5.74s	1.50s	173.69s
-----	-----	-----	-----	-----	-----	-----

stl vektorius

Failas	Kūrimo	Nuskaitymas	Skirstymas	Išvedimas	Rūšiavimas	Viso laikas
kursiokai100k	0.40s	0.95s	0.01s	0.05s	0.03s	10.50s
kursiokai1M	3.92s	9.60s	0.13s	0.52s	0.10s	19.73s
kursiokai10M	47.24s	90.70s	1.15s	4.63s	1.49s	120.21s
-----	-----	-----	-----	-----	-----	-----

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Vector< T >	12
Zmogus	13
Student	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Student	11
Vector< T >	12
Zmogus	13

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

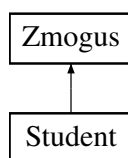
lib.h	15
V1.5vect.h	15

Chapter 5

Class Documentation

5.1 Student Class Reference

Inheritance diagram for Student:



Public Member Functions

- void **test** ()
- **Student** (const **Student** &other)
- **Student** & **operator=** (const **Student** &other)
- **Student** (**Student** &&other)
- **Student** & **operator=** (**Student** &&other)
- void **setVid** (double Vid)
- void **setEgz** (int Egz)
- void **setNd** (const **Vector**< int > &Nd)
- double **getVid** () const
- int **getEgz** () const
- **Vector**< int > & **getNd** ()

Public Member Functions inherited from **Zmogus**

- void **setName** (const string &Name)
- void **setSurn** (const string &Surn)
- string **getName** () const
- string **getSurn** () const

Friends

- istream & **operator>>** (std::istream &in, **Student** &s)
- ostream & **operator<<** (ostream &os, const **Student** &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string **name**
- string **urn**

5.1.1 Member Function Documentation

5.1.1.1 test()

```
void Student::test () [inline], [virtual]
```

Implements [Zmogus](#).

The documentation for this class was generated from the following file:

- V1.5vect.h

5.2 Vector< T > Class Template Reference

Public Member Functions

- T & **operator[]** (size_t index)
- const T & **operator[]** (size_t index) const
- T & **back** ()
- const T & **back** () const
- bool **empty** () const
- void **clear** ()
- T * **begin** ()
- const T * **begin** () const
- T * **end** ()
- const T * **end** () const
- void **push_back** (const T &value)
- void **pop_back** ()
- void **resize** (size_t newSize)
- size_t **getSize** () const
- size_t **getCapacity** () const
- **Vector** (std::initializer_list< T > init)

Static Public Member Functions

- static size_t **getCount** ()

Static Public Attributes

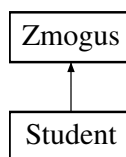
- static size_t **count**

The documentation for this class was generated from the following file:

- V1.5vect.h

5.3 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- void **setName** (const string &Name)
- void **setSurn** (const string &Surn)
- string **getName** () const
- string **getSurn** () const
- virtual void **test** ()=0

Protected Attributes

- string **name**
- string **surn**

The documentation for this class was generated from the following file:

- V1.5vect.h

Chapter 6

File Documentation

6.1 lib.h

```
00001 #ifndef LIB_H
00002 #define LIB_H
00003 #include <iostream>
00004 #include <iomanip>
00005 #include <vector>
00006 #include <chrono>
00007 #include <ctime>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <time.h>
00011 #include <random>
00012 #include <list>
00013 #include <deque>
00014 #include <algorithm>
00015 #include <utility>
00016
00017 using std::string;
00018 //using std::vector;
00019 using std::cout;
00020 using std::cin;
00021 using std::endl;
00022 using std::ifstream;
00023 using std::istringstream;
00024 using std::ofstream;
00025 using std::setw;
00026 using std::left;
00027 using std::fixed;
00028 using std::setprecision;
00029 using std::sort;
00030 using std::swap;
00031 using std::numeric_limits;
00032 using std::streamsize;
00033 using std::ostringstream;
00034 using std::list;
00035 using std::deque;
00036 using std::copy;
00037 using std::ostream;
00038 using std::istream;
00039 using std::to_string;
00040
00041
00042 #endif
```

6.2 V1.5vect.h

```
00001 #ifndef V1_5VECT_H
00002 #define V1_5VECT_H
00003
00004 #include "lib.h"
00005
00006 extern std::chrono::duration<double> pushlaikas; // generavimo laikas
00007 extern std::chrono::duration<double> generationTime; // generavimo laikas
00008 extern std::chrono::duration<double> readTime; // skaitymo laikas
```

```

00009 extern std::chrono::duration<double> sortTime; // skirstymo laikas
00010 extern std::chrono::duration<double> writeTime; // rasymo laikas
00011 extern std::chrono::duration<double> rusiavimoLaikas; // rusiavimo laikas
00012
00013 template <typename T>
00014 class Vector{
00015     private:
00016         T* data; // duomenų masyvas
00017         size_t size; // masyvo dydis
00018         size_t capacity; // talpa
00019
00020     public:
00021         Vector() : data(nullptr), size(0), capacity(0) {} // konstruktorius;
00022
00023         ~Vector(){};
00024
00025         //index operator
00026         T& operator[](size_t index) {
00027             return data[index];
00028         }
00029
00030         const T& operator[](size_t index) const {
00031             return data[index];
00032         }
00033
00034         //back function
00035         T& back() {
00036             return data[size - 1];
00037         }
00038
00039         const T& back() const {
00040             return data[size - 1];
00041         }
00042
00043         //empty function
00044         bool empty() const {
00045             return size == 0;
00046         }
00047
00048         //clear function
00049         void clear() {
00050             size = 0;
00051         }
00052
00053         // begin, end function
00054         T* begin() {
00055             return data;
00056         }
00057
00058         const T* begin() const {
00059             return data;
00060         }
00061
00062         T* end() {
00063             return data + size;
00064         }
00065
00066         const T* end() const {
00067             return data + size;
00068         }
00069
00070         static size_t count; // size == capacity counteris
00071
00072         // push_back funkcija
00073         void push_back(const T& value) {
00074
00075             if (size == capacity) {
00076                 count++;
00077                 size_t new_capacity = (capacity == 0) ? 1 : capacity * 2;
00078                 T* new_data = new T[new_capacity];
00079
00080                 for (size_t i = 0; i < size; ++i) {
00081                     new_data[i] = data[i];
00082                 }
00083
00084                 delete[] data;
00085                 data = new_data;
00086                 capacity = new_capacity;
00087             }
00088
00089             data[size++] = value;
00090         }
00091
00092
00093
00094
00095

```

```

00096         //pop_back
00097         void pop_back() {
00098             if (size == 0) {
00099                 // Vector is already empty
00100                 return;
00101             }
00102             --size;
00103         }
00104
00105         void resize(size_t newSize) {
00106             if (newSize < size) {
00107                 // Shrink: just reduce size, data remains
00108                 size = newSize;
00109             }
00110             else if (newSize > size) {
00111                 if (newSize > capacity) {
00112                     // Need to grow
00113                     size_t new_capacity = newSize;
00114                     T* new_data = new T[new_capacity];
00115
00116                     for (size_t i = 0; i < size; ++i) {
00117                         new_data[i] = data[i];
00118                     }
00119
00120                     delete[] data;
00121                     data = new_data;
00122                     capacity = new_capacity;
00123                 }
00124
00125                 // Default-initialize new elements
00126                 for (size_t i = size; i < newSize; ++i) {
00127                     data[i] = T();
00128                 }
00129
00130                 size = newSize;
00131             }
00132         }
00133
00134         // getteriai
00135         size_t getSize() const {
00136             return size;
00137         }
00138
00139         size_t getCapacity() const {
00140             return capacity;
00141         }
00142
00143         static size_t getCount() {
00144             return count;
00145         }
00146
00147         Vector(std::initializer_list<T> init) : data(nullptr), size(0), capacity(0) {
00148             size = init.size();
00149             capacity = size;
00150             data = new T[capacity];
00151             size_t i = 0;
00152             for (const auto& elem : init) {
00153                 data[i++] = elem;
00154             }
00155         }
00156     };
00157
00158     template <typename T>
00159     bool operator==(const Vector<T>& lhs, const Vector<T>& rhs) {
00160         if (lhs.getSize() != rhs.getSize()) return false;
00161         for (size_t i = 0; i < lhs.getSize(); ++i) {
00162             if (lhs[i] != rhs[i]) return false;
00163         }
00164         return true;
00165     }
00166
00167     template<typename T>
00168     size_t Vector<T>::count = 0;
00169
00170     class Zmogus{
00171     protected:
00172         string name; // vardas
00173         string surn; // pavarde
00174     public:
00175         Zmogus() : name(""), surn("") {} // konstruktorius
00176         virtual ~Zmogus(){}
00177
00178         void setName(const string& Name) {
00179             name = Name;
00180         }
00181
00182         void setSurn(const string& Surn) {

```

```
00183         surn = Surn;
00184     }
00185
00186     string getName() const {
00187         return name;
00188     }
00189
00190     string getSurn() const {
00191         return surn;
00192     }
00193     virtual void test() = 0;
00194 };
00195
00196
00197
00198
00199 class Student : public Zmogus {
00200     private:
00201         Vector<int> nd; // nd rezultatai
00202         int egz; // egzaminu rez
00203         double vid; // galutinis vidurkis
00204
00205     public:
00206
00207         void test(){}
00208
00209         Student() : egz(0), vid(0) {} // konstruktorius
00210
00211         ~Student(){} // destruktorius
00212
00213         // copy constructor dada
00214         Student(const Student& other)
00215             : Zmogus(other), nd(other.nd), egz(other.egz), vid(other.vid){}
00216
00217         // copy assignment operator
00218         Student& operator=(const Student& other){
00219             if(this != &other) {
00220                 name = other.name;
00221                 surn = other.surn;
00222                 nd = other.nd;
00223                 egz = other.egz;
00224                 vid = other.vid;
00225             }
00226             return *this;
00227         }
00228
00229         // move constructor
00230         Student(Student&& other)
00231             : Zmogus(std::move(other)),
00232             nd(std::move(other.nd)),
00233             egz(std::move(other.egz)),
00234             vid(std::move(other.vid))
00235         {
00236             other.egz = 0;
00237             other.vid = 0.0;
00238             other.name = "";
00239             other.surn = "";
00240             other.nd.clear();
00241         }
00242
00243         // move assignment operator
00244         Student& operator=(Student&& other) {
00245             if (this != &other) {
00246                 name = std::move(other.name);
00247                 surn = std::move(other.surn);
00248                 nd = std::move(other.nd);
00249                 egz = std::move(other.egz);
00250                 vid = std::move(other.vid);
00251
00252                 other.egz = 0;
00253                 other.vid = 0.0;
00254                 other.name = "";
00255                 other.surn = "";
00256                 other.nd.clear();
00257             }
00258             return *this;
00259         }
00260
00261         //seteriai
00262         void setVid(double Vid) {
00263             vid = Vid;
00264         }
00265
00266         void setEgz(int Egz) {
00267             egz = Egz;
00268         }
00269 }
```

```

00270     void setNd(const Vector<int>& Nd) {
00271         nd = Nd;
00272     }
00273
00274
00275     //geteriai
00276     double getVid() const {
00277         return vid;
00278     }
00279
00280     int getEgz() const {
00281         return egz;
00282     }
00283
00284     Vector<int>& getNd(){
00285         return nd;
00286     }
00287
00288     friend istream& operator>(std::istream& in, Student& s) {
00289         string name, surname;
00290         Vector<int> nd;
00291         int temp, egz;
00292
00293         in >> name >> surname;
00294         s.setName(name);
00295         s.setSurn(surname);
00296
00297         while (in >> temp) {
00298             nd.push_back(temp);
00299         }
00300
00301         if (!nd.empty()) {
00302             egz = nd.back();
00303             nd.pop_back(); // Remove last element, which is egz
00304             s.setEgz(egz);
00305             s.setNd(nd);
00306         }
00307
00308         in.clear(); // Clear stream state if end of line or error
00309         return in;
00310     }
00311
00312     // isvesties operatorius
00313     friend ostream& operator<(ostream& os, const Student& s){
00314         os << left << setw(20) << s.name << setw(16) << s.surn << s.vid << endl;
00315         return os;
00316     }
00317
00318 };
00319
00320 extern Vector<Student> BadStudents;
00321 extern Vector<Student> GoodStudents;
00322
00323 void skaitom(int pasirinkimas, string A[], string B[]);
00324 void vidurkis();
00325 void mediana();
00326 void spausdinam(char a);
00327 void generuojam(string b, int n);
00328 void pushTestas(int n);
00329 void rusiuojam2(); // skaidymas per puse
00330 void rusiuojam1(); // skaidymas is vieno konteinerio i du
00331 bool compareByName(const Student& a, const Student& b);
00332 bool compareBySurname(const Student& a, const Student& b);
00333 bool compareByVid(const Student& a, const Student& b);
00334 void testas();
00335
00336 #endif

```


Index

README, [1](#)

Student, [11](#)
test, [12](#)

test
Student, [12](#)

Vector< T >, [12](#)

Zmogus, [13](#)